

网络安全智能体 Benchmark

数据构造调研报告

论文逐篇阅读 · 构造流程 · 真值与验证 · 数据偏差

覆盖对象

CyberGym-E2E、BountyBench、SEC-bench Pro、ExploitGym、AgentCyberRange、VulnGym、CyberGym、ExploitBench、CVE-Bench、AutoPenBench、CyBench、NYU CTF Bench

资料截止日期：2026 年 7 月 28 日

用途：Coding Agent / Cyber Agent 评测选型与训练环境设计

目录与阅读说明

阅读口径

本文聚焦“benchmark 数据如何构造”，而不是复述模型排行榜。每一章均区分原始数据源、样本筛选、历史环境重建、人工或自动标注、动态验证、最终落盘对象和已知偏差。无法从论文或仓库核实的内容不作推断。

目录

- 1. [总体结论](#)
- 2. [横向比较](#)
- 3. [CyberGym-E2E](#)
- 4. [BountyBench](#)
- 5. [SEC-bench Pro](#)
- 6. [ExploitGym](#)
- 7. [AgentCyberRange](#)
- 8. [VulnGym](#)
- 9. [CyberGym](#)
- 10. [ExploitBench](#)
- 11. [CVE-Bench \(Web 利用版\)](#)
- 12. [AutoPenBench](#)
- 13. [CyBench](#)
- 14. [NYU CTF Bench](#)
- 15. [跨 benchmark 的构造方法总结](#)
- 16. [参考文献与项目链接](#)

证据等级

A: 论文 + 公开仓库	构造方法、数据格式和执行环境可交叉核验，证据最完整。
B: 论文为主	论文详细说明构造流程，但部分数据或环境未完整公开，复核范围受限。
C: 仓库为主	尚无正式论文，仅依据 README、schema、数据卡和发布记录总结。

两个重要版本说明

SEC-bench Pro 在 2026 年 7 月 20 日发布 v2，规模由 v1 的 183 个浏览器漏洞扩展为 344 个 V8、SpiderMonkey 与 Linux 内核实例；本文以 v2 为准。VulnGym 0.1.4 的配套论文仍在撰写，因此该章明确标为“仓库资料总结”。

1. 总体结论

这 12 个 benchmark 并不是同一种数据集的不同规模版本，而是沿着“发现、复现、利用、修复、横向移动”五类能力形成了互补谱系。最关键的差异不在任务名称，而在样本起点、可见信息和动态真值。

1.1 六类数据构造范式

补丁差分型 CyberGym、CyberGym-E2E：从 OSS-Fuzz / ARVO 历史漏洞出发，定位修复提交，重建修复前后版本，并以崩溃差分和功能测试形成动态真值。
公开漏洞报告型 SEC-bench Pro、CVE-Bench：从 issue、CVE、advisory、PoC 与修复提交出发，人工或 Agent 重建历史环境，再用脆弱 / 修复镜像双向验证。
漏洞奖励生命周期型 BountyBench：把同一个真实 bug bounty 转换为 Detect、Exploit、Patch 三类任务，并保留系统快照、数据库状态、运行时不变量与经济奖励。
已知触发到利用升级型 ExploitGym、ExploitBench：样本起点不是“发现漏洞”，而是已给定触发器、漏洞描述或补丁；重点构造分级利用目标和抗作弊动态 oracle。
靶场攻击链型 AgentCyberRange、AutoPenBench：构造可交互网络与多服务环境，通过旗标、状态变化、主机证明文件或里程碑判断攻击是否完成。
代码路径 / CTF 归档型 VulnGym 以入口点到敏感操作的数据流路径作为真值；CyBench 与 NYU CTF Bench 则把公开比赛题 docker 化、标准化并补充中间目标。

1.2 选择 benchmark 时应优先检查什么

- 是否要求从真实用户入口发现漏洞，还是已经给定漏洞描述、补丁、PoC 或触发输入。
- ground truth 是静态位置标签、原始 PoC、崩溃签名、应用状态变化，还是完整利用能力阶梯。
- 评测是否同时验证 vulnerable、fixed、latest 三个版本，能否排除“撞到另一个 bug”的误判。
- 环境是否包含真实构建系统、数据库、服务依赖、多主机网络和安全缓解机制。
- 数据是否公开到足以复现，是否存在公开 writeup 或补丁导致的训练污染。

面向你给出的岗位能力要求

若要评估“真实仓库漏洞发现 + PoC + 修复”，CyberGym-E2E、BountyBench、SEC-bench Pro 最接近。若重点考察高级利用，ExploitGym 与 ExploitBench 更强，但它们并不测试从零发现漏洞。若重点考察 Web、内网、后渗透和跨主机攻击链，AgentCyberRange 的覆盖最完整。

2. 横向比较

2.1 数据来源与任务起点

Benchmark	主要原始来源	Agent 起点	主要真值
CyberGym-E2E	OSS-Fuzz / ARVO + 修复提交	仅仓库与任务目标	PoC 差分 + 功能测试 + 补丁
BountyBench	公开 bug bounty 报告	按 Detect / Exploit / Patch 变化	运行时不变量 + exploit + patch
SEC-bench Pro	浏览器 issue、advisory、kernelCTF、syzbot	报告摘要与目标约束，无参考 PoC	三镜像重放 + LLM 归因
ExploitGym	真实漏洞及已知触发输入	给定 PoV / trigger	分级安全影响 + 动态旗标
AgentCyberRange	真实应用漏洞 + 人工构造企业网	URL / 入口 IP，提示分级	PoC 定位 + 主机证明文件
VulnGym	GitHub Security Advisories	完整项目白盒源码	入口点 + 敏感操作 + 路径
CyberGym	OSS-Fuzz / ARVO	漏洞文本 + 修复前代码	修复前崩溃、修复后不崩溃
ExploitBench	2024 年后 V8 N-day	描述 + patch diff	16 项能力旗标
CVE-Bench	NVD 关键 Web CVE	零日 / 一日两种信息	应用状态型 grader
AutoPenBench	课程靶题 + Metasploit CVE	Kali / 目标主机	flag + 手工 milestone
CyBench	四个专业 CTF 比赛	题面 + 附件 + 容器	flag + 人工子任务
NYU CTF Bench	CSAW CTF 2017-2023	题面 + 可见附件	隐藏 flag

2.2 公开性与复现成本

“公开”不能简单理解为下载一个 JSON 文件即可评测。真实漏洞 benchmark 的主要成本通常在历史依赖、构建工具链、服务状态、数据库初始化和动态验证。以下是从构造角度的相对判断。

低至中等：VulnGym、CyBench、NYU CTF Bench	主要是结构化标注或已 docker 化 CTF；单个实例相对轻。
中等：AutoPenBench、CVE-Bench、BountyBench	包含服务、数据库与网络，但项目规模或系统数有限。
高：CyberGym、CyberGym-E2E、SEC-bench Pro	大量历史 C/C++、浏览器或内核构建，依赖修复前后镜像。
很高：ExploitGym、ExploitBench、AgentCyberRange	需要多种缓解配置、利用分级 oracle 或多主机企业网络。

3. CyberGym-E2E

面向真实开源仓库的“发现 - PoC - 修复”全生命周期 benchmark

论文 / 资料版本	arXiv:2606.04460, 2026-06; 当前论文规模 920 个漏洞
任务定位	要求 Agent 从仓库级环境中发现漏洞、生成可执行 PoC, 并提交不破坏功能的补丁。
数据规模	920 个真实漏洞, 覆盖 139 个开源项目。
样本基本单位	一个漏洞实例, 包含修复前仓库、可复现漏洞、原始修复和开发者功能测试。
证据等级	A: 论文 + 公开 benchmark 基础设施。
一句话理解 它不是简单把 CyberGym 的 PoC 任务再加一个 patch 字段, 而是额外构造了可运行的开发者测试套件, 使“修复漏洞但不破坏功能”成为可自动验证的端到端任务。	

3.1 数据构造流水线

1

原始漏洞汇集: 以 OSS-Fuzz 公开漏洞及 ARVO 的可复现制品为主要来源。ARVO 提供历史源码、容器和崩溃输入, 但原始制品并不包含适合评测补丁正确性的项目级功能测试。

2

修复提交定位与清洗: 围绕 OSS-Fuzz 标记的修复时间, 对提交历史进行二分或区间搜索, 定位从“仍可崩溃”到“不再崩溃”的修复点; 删除信息不足、一次提交混合多个独立问题、补丁跨度过大或修复归因不清的候选。

3

脆弱 / 修复版本重建: 分别构建修复前与修复后版本, 执行原始 PoC, 要求脆弱版本稳定触发目标崩溃, 而补丁版本不再出现相同故障。历史项目无法在原始旧系统直接运行时, 构造流水线会迁移依赖或基础镜像。

4

功能测试自动发现: 让 coding agent 在已经应用原始补丁的容器中分析项目构建系统, 寻找项目自带的单元测试、集成测试或回归测试, 并生成统一的 build / test 脚本。此步骤的目标是把“软件仍然能用”转化为可执行 oracle。

5

专家复核与返工: 安全与工程专家检查构建脚本、测试日志、覆盖范围和实例元数据。脚本不可靠、测试过弱或环境不稳定时重新提示 Agent 修复; 无法达到可重复运行标准的实例被排除。

6

发布统一任务包: 把修复前环境、参考 PoC、崩溃证据、原始补丁、功能测试和任务元数据封装为统一实例, 使发现、PoC 与 patch 三个阶段可以分别评测, 也可以要求端到端一次完成。

3.2 落盘对象与数据格式

每个实例的核心不是一条漏洞描述，而是一组可执行制品。论文强调以容器和脚本固定历史构建环境，并保存能够区分安全性与功能性的两套证据。

- 修复前源码与可构建环境，以及对应的修复后参考版本。
- 原始或标准化 PoC、崩溃日志、目标 sanitizer / 错误类型。
- ground-truth patch 与补丁元数据。
- 自动发现并经人工审核的开发者功能测试和统一执行脚本。
- 项目、提交、文件规模、补丁规模与执行结果等结构化元数据。

3.3 真值、评分与质量控制

安全性与功能性采用分离的动态 oracle。安全 oracle 判断 PoC 是否在脆弱版本触发并被补丁抑制；功能 oracle 判断补丁后的项目测试是否仍然通过。参考 PoC 还用于确认 Agent 提交的修复针对的是预期漏洞，而不是通过禁用功能或修改测试绕过。

- 脆弱版本必须复现目标漏洞，修复版本必须抑制该漏洞。
- Agent patch 必须通过项目功能测试，不能只让崩溃输入失效。
- 专家审核测试的真实性和覆盖，不接受仅检查进程退出码的空测试。
- 论文报告了多级筛选漏斗：候选经过修复定位、环境复现、测试构造和专家验收后显著缩减；最终规模为 920。

3.4 构造偏差与使用限制

数据仍主要继承 OSS-Fuzz 的分布，因此偏向 C/C++、内存安全、可由崩溃输入观察的问题。开发者测试能够减少“过拟合 PoC”的补丁，但无法证明语义完全等价；同一项目的历史漏洞之间也存在相关性和潜在训练污染。

- 不覆盖大量业务逻辑、访问控制、配置和分布式系统漏洞。
- 从已公开漏洞反向构造任务，不能完全等价于真实零日发现。
- 测试套件强度由项目现有测试和 Agent 发现能力共同决定。

对训练与评测设计的启示

构造端到端安全 Agent 环境时，必须同时保存“攻击证据”和“正常功能证据”。仅用原始 PoC 作为奖励，很容易训练出删除代码、拒绝所有输入或针对单个样本打补丁的 reward hacking。

主要依据：[CyberGym-E2E: Scalable Real-World Benchmark for AI Agents' End-to-End Cybersecurity Capabilities](#)

4. BountyBench

以真实 bug bounty 为中心的 Detect / Exploit / Patch 生命周期 benchmark

论文 / 资料版本	arXiv:2505.15216, NeurIPS 2025; 论文与项目环境
任务定位	在持续演化的真实 Web / 服务代码库中评估漏洞发现、利用和修复，并用真实奖金衡量经济影响。
数据规模	25 个真实系统，40 个公开 bug bounty; 3 类核心任务共 120 个基础任务，Detect 另有多档信息设置。
样本基本单位	一个“系统快照 + 漏洞 + exploit + verifier + patch + 不变量”的生命周期对象。
证据等级	A: 论文 + 公开框架 / 环境。
一句话理解 BountyBench 的构造中心不是 CVE 条目，而是一个随提交演化的真实系统；同一漏洞被复用为发现、利用和修复三种任务，并由运行时状态和代码不变量共同判分。	

4.1 数据构造流水线

1

真实奖励报告收集：收集开源 GitHub 项目中已经公开披露的 bug bounty 报告，保留漏洞类别、报告描述、时间线和真实奖金。论文中的 40 个奖励覆盖多个 OWASP Top 10 风险，且多数披露于 2024-2025 年。

2

系统快照与版本链整理：围绕漏洞出现、披露和修复时间建立仓库快照。一个系统可包含多个提交和多个漏洞，而不是为每个漏洞复制一个孤立仓库；这样可以利用历史快照判断 exploit 是否只在特定版本有效。

3

手工重建完整运行环境：研究人员安装语言运行时和依赖，启动应用、数据库及辅助服务，并对数据库进行数据灌装。Agent 位于带代码库与安全工具的容器中，目标应用可由多个网络互联容器组成。

4

人工复现与参考制品编写：逐个手工复现 bounty 报告，编写可靠 exploit、自动 verifier 和参考 patch；同时定义应用应保持的运行时不变量及代码不变量，例如账号权限、数据库内容、关键接口行为和不可删除的安全检查。

5

任务实例化：Detect 任务隐藏具体漏洞，要求发现新的安全问题；Exploit 给定目标漏洞并要求产生实际影响；Patch 给定漏洞并要求修复。Detect 通过不同信息量插值，从接近零日的宽泛目标逐步变为更具体的提示。

6

交叉审核与持续集成：构造者之间交叉审阅 exploit、patch 和不变量；CI 重建环境并执行参考攻击与修复。论文还人工检查运行日志，排除环境偶发错误或 verifier 只匹配表面字符串的情况。

4.2 落盘对象与数据格式

BountyBench 将系统建模为多个关联对象，核心落盘内容既包括代码，也包括环境状态和随时间变化的版本关系。

- 真实代码库及多个时间快照，应用 / 数据库 / 支撑服务容器。
- 漏洞报告、真实奖金、OWASP 类别和 Detect 信息级别。
- 可执行 exploit、定位用 Detect Indicator、动态 verifier。
- 参考 patch、运行时不变量和代码不变量。
- Agent 容器、目标运行时容器及隔离的隐藏验证容器。

4.3 真值、评分与质量控制

Detect 的成功不能只依赖 Agent 自述。系统要求找到可被 verifier 定位的运行时异常，或给出在当前快照有效、在至少一个修复快照失效的 exploit。Exploit 需要在目标快照产生规定影响；Patch 需要使 exploit 失败，同时保持功能和不变量。

- Detect Indicator 将动态结果定位到具体漏洞，解决“系统里本来就有多个漏洞”的归因问题。
- 历史快照提供差分证据，可防止把正常功能或无关错误误判为新漏洞。
- Patch 同时执行安全回归与正常功能检查。
- 参考 exploit 与 verifier 被隐藏，降低硬编码答案和修改评分器的风险。

4.4 构造偏差与使用限制

40 个 bounty 的环境重建高度依赖人工，规模不如自动化 OSS-Fuzz 系列；公开报告、奖金和补丁可能已经进入模型训练数据。同一系统内多个任务共享仓库和环境，统计上并非完全独立。

- Web / 服务应用占比高，对浏览器内核和二进制高级利用覆盖有限。
- 真实奖金可表示历史经济影响，但不等价于漏洞在新环境中的当前风险。
- Detect 的信息梯度虽能控制难度，仍以已知漏洞为 ground truth。

对训练与评测设计的启示

若希望训练同时具备攻防能力的 Agent，可把同一漏洞的数据对象设计为状态机：漏洞出现、被发现、被利用、被修复。这样能复用环境，并用版本链和不变量提供比单一 flag 更可靠的奖励。

主要依据：[BountyBench: Dollar Impact of AI Agent Attackers and Defenders on Real-World Cybersecurity Systems](#)

5. SEC-bench Pro

V8、SpiderMonkey 与 Linux 内核上的长程 PoC 合成 benchmark

论文 / 资料版本	arXiv:2605.26548v2, 2026-07-20; 本文采用扩展后的 v2
任务定位	在超大复杂代码库中, 根据公开报告给出的有限触发约束审计源码并重新合成 PoC。
数据规模	344 个经双 oracle 验证的实例: V8 103、SpiderMonkey 104、Linux 内核 137。
样本基本单位	一个公开安全报告对应的“脆弱 / 修复 / 最新”三镜像任务。
证据等级	A: 论文 v2 + GitHub 仓库。
一句话理解 SEC-bench Pro 用 Agent 自动重建历史浏览器和内核漏洞环境, 再用脆弱、修复、最新三个版本重放提交的 PoC; 其重点是深路径触发合成, 而不是在给定 fuzz harness 上变异字节。	

5.1 数据构造流水线

1

项目化报告采集: 为 Chromium Issue Tracker、Mozilla Bugzilla、MFSA、Pwn2Own、CISA KEV、kernelCTF 和 syzbot 编写适配器, 抽取标题、描述、严重性、PoC、修复提交和赏金等字段。缺少具体 PoC 或 linked fix 的报告暂不进入后续构造。

2

目标级筛选: V8 侧从固定的 P0 / P1 安全报告、sandbox 与 WebAssembly 查询开始; SpiderMonkey 侧要求 sec-high / sec-critical 并扩展安全公告; Linux 侧要求 kernelCTF / syzbot 中具有 CVE、触发器和上游修复。

3

Agent 驱动环境重建: coding agent 接收原始报告、参考 PoC、目标二进制、允许的 flags 和预期错误类型, 在隔离容器中自行确定工具链、依赖和构建参数, 生成 sanitizer / debug 浏览器引擎, 或可在 QEMU 中启动的 KASAN 内核。

4

真实崩溃证据固化: Agent 执行参考 PoC, 反复调整构建配置, 直到捕获与报告一致的 crash signature。流水线要求保存真实 stderr 或内核串口日志, 并明确禁止伪造输出。

5

双构造 oracle 验证: 脆弱镜像多次执行 PoC, 至少一次必须匹配项目特定错误分类; 修复镜像应用 linked patch 后以相同配置重建, 多次执行均必须落入“被阻断”类别。仅通过两侧 oracle 的实例进入数据集。

生成三镜像评测任务：除 vulnerable 与 fixed 外，再加入 latest upstream 镜像。评测时隐藏参考 PoC 和根因位置，只给目标源码范围、漏洞类型和错误类型；Agent 必须审计源码并保存候选 PoC。

5.2 落盘对象与数据格式

每个实例采用统一 bundle，允许不同项目共享评分流程，同时保留项目特定的构建与崩溃分类。

- vulnerable、fixed、latest 三个 Docker 镜像。
- 源码、插桩二进制、原始 bug report、canonical PoC、upstream patch。
- 在脆弱镜像中真实捕获的错误输出或 KASAN 串口日志。
- 结构化 metadata：目标路径、允许 flags、错误类型、漏洞类别、执行权限。
- 浏览器 JavaScript PoC 或 Linux C / syscall PoC 的统一提交目录。

5.3 真值、评分与质量控制

评测 grader 将每个候选 PoC 在三镜像上重放并最多重试三次，收集退出码、stdout 和 stderr。LLM judge 再结合目标文件、漏洞类型与错误类型，将结果分为 verified、unsure、illegal，以区分目标漏洞、补丁旁路后的 sibling bug、无关崩溃和基础设施错误。

- 三镜像比传统 vulnerable / fixed 差分提供更多归因证据。
- Linux PoC 记录实际 uid，要求用户可达漏洞不能依赖 root。
- 预筛禁止测试专用原语，防止 Agent 使用不符合真实环境的内部接口。
- 论文 v2 报告 LLM judge 相对人工 ground truth 达到 99.1% precision、97.2% recall。

5.4 构造偏差与使用限制

该 benchmark 仍从“已经给出 PoC 与修复的公开报告”构造，Agent 接收漏洞类别和期望错误类型，因此不是完全无提示零日发现。LLM judge 提高归因能力，但也引入模型依赖和审计成本。

- 大型镜像、历史工具链、QEMU 和 sanitizer 使复现成本很高。
- 浏览器和内核分布强，难以代表 Web 业务逻辑或云配置漏洞。
- v1 论文摘要常被索引为 183 个浏览器漏洞，引用时必须注明 v2 的 344 个实例。

对训练与评测设计的启示

对复杂代码库，二值“修复前崩溃、修复后不崩溃”可能把 sibling bug 判错。更可靠的设计是保留最新版本、目标文件和错误类别，并把执行证据与语义归因分层。

主要依据：[SEC-bench Pro: Can Language Models Solve Long-Horizon Software Security Tasks? \(v2\)](#)

6. ExploitGym

从已知漏洞触发器逐步升级为真实安全影响的大规模利用 benchmark

论文 / 资料版本	arXiv:2605.11086, 2026-05
任务定位	给定能触发漏洞的 PoV, 要求 Agent 进一步获得读写、控制流或权限等可验证影响。
数据规模	898 个实例: userspace 520、V8 185、Linux 内核 193; 并提供不同缓解配置。
样本基本单位	一个真实漏洞在特定安全缓解设置下的利用任务。
证据等级	A: 论文 + 容器化 benchmark。
一句话理解 ExploitGym 刻意把“发现漏洞”移出任务边界, 统一从一个已能触发故障的输入开始, 以便单独测量 Agent 能否跨过从 crash 到 exploit 的难点。	

6.1 数据构造流水线

1

用户态漏洞来源: 主要从 CyberGym / OSS-Fuzz 继承可复现漏洞, 并补充 OSV 中非 fuzz 类漏洞。对缺少现成触发器的 OSV 条目, 使用 Claude Code / Opus 生成 PoV, 再由人工确认。

2

去除调试依赖并恢复生产构建: 重新构建不依赖 sanitizer 的用户态程序, 修复陈旧依赖和历史工具链, 使触发器在更接近真实发布环境的二进制上仍有效。最终保留 520 个用户态实例, 覆盖 161 个项目。

3

V8 漏洞筛选与触发器恢复: 收集 2024 年后、默认启用 heap sandbox 背景下的 Chromium / V8 漏洞。ClusterFuzz 报告通过补丁或测试恢复 PoV, 人工报告采用附件中的 PoV; 从 403 个候选中构建并验证 185 个。

4

Linux 内核样本整理: 从 kernelCTF 与 syzbot 获取真实内核漏洞。kernelCTF 的完整 exploit 被人工与 Agent 协作缩减为最小 PoV, syzbot 使用 C reproducer; 对同一根因去重并选择最早上游报告, 最终得到 193 个。

5

安全缓解配置化: 为用户态切换 ASLR / PIE 和 stack canary, 为 V8 切换 ASLR / heap sandbox, 为内核切换 KASLR / user namespace 等设置, 形成同一漏洞在不同防护强度下的对照任务。

6

分级目标与容器封装: 给定初始 PoV, 要求 Agent 逐步扩展到更强的安全影响。所有目标按领域封装为可复现容器或虚拟机, 并统一暴露提交与验证接口。

6.2 落盘对象与数据格式

每个实例包含一个确认可触发的起始输入和与领域相匹配的目标环境。重点落盘的是“利用进度可以被动态观察”的任务，而不是静态 exploit 文本。

- 漏洞版本源码 / 二进制、起始 PoV 和运行脚本。
- 用户态、V8、内核三类领域特定环境。
- 开启 / 关闭关键缓解机制的配置。
- 分级安全影响目标、动态旗标和验证器。
- 用于阻止修改 grader、伪造输出和读取秘密的隔离边界。

6.3 真值、评分与质量控制

ExploitGym 使用高权限的动态 grader 和 agent-as-judge 机制验证真实影响。Agent 不能只打印预期字符串，而必须在隔离目标中触发指定能力；不同领域使用相应的内存、控制流、文件或权限证据。

- 初始 PoV 已验证，因此评测把失败原因集中到利用开发，而非漏洞定位。
- 同一漏洞在多种缓解配置下执行，可估计 ASLR、sandbox、KASLR 等对 Agent 的真实阻碍。
- grader 与 Agent 权限分离，并使用运行时秘密或特权状态降低 reward hacking。

6.4 构造偏差与使用限制

它不评价漏洞发现，也不要求从真实用户入口建立完整攻击面；给定 PoV 会显著降低任务不确定性。不同领域的“成功影响”并不完全等价，将 898 个实例汇总为单一成功率时需要谨慎。

- 部分 OSV PoV 由模型生成后再人工验证，可能偏向模型容易合成的触发模式。
- 历史漏洞和公开 exploit 仍存在训练污染风险。
- 容器 / 虚拟机内利用与真实产品完整沙箱链仍有差距。

对训练与评测设计的启示

若研究目标是强化学习中的 exploit skill，应把漏洞发现与利用拆开，并提供稳定起始 PoV、逐级 reward 和抗作弊 verifier；否则稀疏的“最终 RCE”奖励难以区分搜索过程中的有效进展。

主要依据：[ExploitGym: Can AI Agents Turn Security Vulnerabilities into Real Attacks?](#)

7. AgentCyberRange

真实 Web 漏洞与企业式多主机后渗透攻击链 benchmark

论文 / 资料版本	arXiv:2606.14295, 2026-06
任务定位	覆盖外部 Web 初始访问和进入内网后的侦察、横向移动、提权、凭证利用与多主机攻击链。
数据规模	WebExploitBench: 15 个真实应用、110 个漏洞; PostExploitBench: 8 个 range、156 台主机、43 个链节点、113 个诱饵。
样本基本单位	Web 部分以一个应用漏洞为实例; 后渗透部分以一条需要跨主机完成的攻击链节点为实例。
证据等级	A: 论文 + CAGE / cyber range 代码与环境。
一句话理解 它把 benchmark 从“单容器拿 flag”推进到多网段、多角色、多诱饵的企业环境，并分别构造外部入口和后渗透两阶段任务。	

7.1 数据构造流水线

1

真实应用选择：由具有五年以上经验的安全专家选择 CMS、博客、电商、BI、AI 与企业应用，覆盖 Python、PHP、Java 等技术栈。要求漏洞可通过黑盒交互触达，排除必须知道不可见实现细节才能利用的样本。

2

三类 Web 漏洞组合：收集已披露 one-day，加入负责任披露的 zero-day，并在真实应用安全 workflow 内植入 synthetic 漏洞。论文共构造 18 个 zero-day、56 个 one-day、36 个 synthetic，覆盖 17 类漏洞。

3

提示难度分级：Web L0 只给目标 URL，L1 指出脆弱 URL，L2 再给漏洞类型；后渗透 L0 只给入口 IP，L1 给网络拓扑，L2 给 CVE 或弱点细节。相同环境由提示量而非更换漏洞来控制难度。

4

企业网络拓扑构造：每个 range 用多个 Docker 容器和隔离虚拟网络构成公开区、业务子网和深层内网，通常约 20 台主机、3-7 个 /24 子网。只有多网卡 pivot host 可以桥接不同网段。

5

攻击链与诱饵布置：每个 range 设置 3-6 个真实链节点，并加入大量 decoy 和支撑服务。链条覆盖横向移动、提权、凭证发现 / 复用、文件与配置发现、服务跳板、数据库、CI / 仓库 / 代码、SMB、持久化和规避等技术。

6

防守动态与编排：加入服务重启、防御机制、honeypot 和由 Agent 模拟的 defender。CAGE 负责部署、重置、适配不同 Agent、记录轨迹和执行验证，使同一 range 可重复评测。

7.2 落盘对象与数据格式

Web 与后渗透数据共享编排框架，但落盘对象不同：Web 强调目标应用、漏洞位置和 PoC；后渗透强调网络拓扑、主机镜像、凭证与攻击链状态。

- 15 个真实应用及其 110 个漏洞配置、参考 PoC 和提示级别。
- 8 套多网段 range 的 Docker 主机、网络、路由和访问控制。
- 156 台主机的服务、账号、凭证、文件、权限和链节点配置。
- 43 个有效攻击链节点与 113 个 decoy。
- CAGE 的部署、重置、日志、结果收集与 verifier。

7.3 真值、评分与质量控制

Web verifier 先观察真实安全结果，再解析 Agent PoC 的 URL 组件，与参考 PoC 的目标位置比较，避免同类型但不同端点的漏洞被错误归因。后渗透任务要求在指定主机路径写入证明文件；提权节点使用只有 root 可写的位置，grader 从外部轮询主机。

- 验证的是目标系统状态，不依赖 Agent 最终文字报告。
- 每个链节点可独立计分，使长攻击链具有中间进度信号。
- 隔离网络和唯一 pivot 限制保证横向移动必须实际发生。
- zero-day 经负责任披露处理，避免把未修复漏洞直接作为公开攻击教程。

7.4 构造偏差与使用限制

多主机靶场高度依赖专家设计，拓扑和凭证分布可能带有构造者偏好。Docker 网络不完全等价于真实企业的 AD、EDR、云 IAM 和复杂流量；synthetic 漏洞虽嵌入真实应用，仍不是自然产生的历史漏洞。

- 数据规模按节点和漏洞统计，不能直接与“仓库数”或“CVE 数”比较。
- 提示级别会显著改变任务性质，报告结果时必须注明 L0 / L1 / L2。
- 环境维护成本高，镜像更新可能破坏历史可复现性。

对训练与评测设计的启示

评测后渗透 Agent 时，应把网络可达性、凭证、权限和主机状态作为一等数据对象；单个最终 flag 无法解释 Agent 卡在侦察、pivot、提权还是目标定位。

主要依据：[AgentCyberRange: Benchmarking Frontier AI Systems in Realistic Cyber Ranges](#)

8. VulnGym

以可达入口点和代码语义证据链为真值的白盒项目级漏洞检测数据集

论文 / 资料版本	GitHub 数据集 v0.1.4，发布于 2026-06-26；配套论文尚在撰写
任务定位	评估 Agent 在完整真实仓库中识别漏洞入口、关键危险操作和可达代码路径的能力。
数据规模	184 个 GitHub Advisory，408 个入口点样本，38 个项目 / 23 个仓库；393 / 408 已人工审核。
样本基本单位	一个 advisory 下的一个 reachable entry point；同一漏洞可有多个入口点。
证据等级	C：公开仓库、schema、README 与数据卡；尚无正式论文。
一句话理解 VulnGym 把“找到漏洞”定义为定位真实入口点和关键危险操作，而不是要求 exploit；其 ground truth 是可核对的源码行与有序语义路径。	

8.1 数据构造流水线

1

Advisory 样本收集：当前仓库说明数据主要来自 GitHub Security Advisories，并偏向近期、高星真实开源项目。数据同时覆盖业务逻辑类和传统污点 / 内存 / 注入类问题，其中仓库统计显示业务逻辑占比较高。

2

版本与仓库绑定：每个报告绑定 repository URL 和 vulnerable commit，避免只保留脱离历史版本的函数片段。advisory 级记录与入口点级记录通过稳定 ID 关联。

3

入口点识别：为每个漏洞枚举从外部或业务边界可达的 entry_point。一个 advisory 可能对应多个 HTTP handler、RPC 方法、CLI / 文件解析入口，因此被拆为多个 entries。

4

关键操作与路径标注：标注 critical_operation，并给出从入口到关键操作的有序 trace。每个节点包含文件路径、行号、原始代码和可选说明，强调可由代码审计核验的语义证据链。

5

自动标注与人工审核：初始注释可由自动工具生成；人工审阅后设置 verify=1。v0.1.4 中 393 / 408 个入口已审核，178 / 184 个 advisory 至少包含一个已验证入口。未审核标注明确视为可能变化。

6

发布一致性检查：发布脚本检查 ID 和 join、URL / commit 格式、节点 schema、行号为正、排序稳定、内部字段不泄漏等不变量，保证 reports.jsonl 与 entries.jsonl 可稳定用于评测。

8.2 落盘对象与数据格式

VulnGym 主要是结构化静态数据，而非预构建靶场。当前仓库的核心文件是 advisory 级与入口点级两个 JSONL。

- reports.jsonl: 每个 GitHub advisory 一条记录。
- entries.jsonl: 每个 reachable entry point 一条记录。
- repo_url、commit、entry_point、critical_operation、trace、verify 等字段。
- 文件路径、行号、verbatim code 和语义描述。
- 评测脚本与 release invariant 检查。

8.3 真值、评分与质量控制

默认评测只要求工具输出 repo_url、commit、entry_point 和 critical_operation；trace 可选且当前评分忽略。路径正规化后进行精确文件匹配，行号允许正负 5 行误差，并严格检查从入口到危险操作的方向。主要指标是 advisory-level recall，其次是 entry-level recall。

- 一个 advisory 只要命中至少一个已验证入口即可计入 advisory recall。
- entry-level recall 衡量对多个入口覆盖是否完整。
- 当前 evaluator 不惩罚过量报告，因此并非 precision-aware。
- 静态位置标签可以快速批量评测，但不验证候选漏洞是否可通过运行时 PoC 触发。

8.4 构造偏差与使用限制

最重要的限制是尚无正式论文，自动标注、项目筛选和人工审核协议尚未完整公开；另外当前指标只看 recall，不处罚误报，容易鼓励输出大量候选位置。

- 未提供统一可执行环境和动态 exploit / PoC oracle。
- ± 5 行容差适合定位，但无法判断根因解释是否正确。
- 多入口拆分会造成同一 advisory 内样本相关。

对训练与评测设计的启示

代码审计 benchmark 可以把入口、敏感操作和证据链作为可解释的中间监督，但若用于强化学习，应补充误报惩罚和动态可达性验证，否则 recall-only 奖励容易被“全仓库撒点”利用。

主要依据：[Tencent VulnGym repository and README \(v0.1.4\)](#)

9. CyberGym

基于 OSS-Fuzz 历史漏洞的大规模真实 PoC 生成 benchmark

论文 / 资料版本	arXiv:2506.02548v3, ICLR 2026; 论文当前版本 2026-03
任务定位	根据漏洞文本描述和修复前代码库生成能复现真实漏洞的 PoC。
数据规模	1,507 个真实漏洞，覆盖 188 个软件项目；公开披露时间约为 2017-01-01 至 2025-04-21。
样本基本单位	一个 OSS-Fuzz 漏洞的修复前 / 后可执行文件、描述、PoC 和补丁。
证据等级	A: 论文 + 开源 benchmark / ARVO 制品。
一句话理解 CyberGym 的关键贡献是把 OSS-Fuzz 漏洞历史自动转成“给描述和仓库，生成 PoC”的动态任务，并用修复前后差分而不是静态答案判分。	

9.1 数据构造流水线

1

OSS-Fuzz / ARVO 数据汇集：从 OSS-Fuzz issue 生命周期、崩溃输入和构建信息出发，并复用 ARVO 的历史可复现制品。1,368 个实例来自 ARVO 截止 2024-07-31 的集合，另补充 139 个更新漏洞。

2

修复提交搜索：在 OSS-Fuzz 标记的 fixed date 之前对提交进行二分搜索，找到最后一个仍会崩溃的版本和第一个不再崩溃的版本，据此确定 vulnerable commit、fixed commit 与 ground-truth patch。

3

容器化 sanitizer 复现：在项目对应的 OSS-Fuzz / Docker 环境中编译插桩二进制，运行原始 PoC，确认修复前触发 sanitizer 或目标错误，修复后不再触发。

4

任务描述构造：从提交信息和 issue 元数据生成漏洞描述，再使用 GPT-4.1 重述为适合 Agent 的任务文本。描述与代码库一起作为主要输入，PoC 和补丁在评测时隐藏。

5

描述质量筛选：自动移除无信息描述、一次混合多个独立问题的描述，并使用 few-shot GPT judge 判断描述是否足以对应一个明确漏洞。论文抽样 300 条人工核验，报告约 96% precision。

6

歧义与重复处理：根据崩溃栈相似度和修复差分处理重复或归因不清的实例，最终保存可重复执行的修复前 / 后二进制和 ground truth。

9.2 落盘对象与数据格式

CyberGym 的实例围绕可重放崩溃组织，典型数据对象包含描述、历史代码、两个版本的可执行环境以及参考 PoC / patch。

- 修复前源码与代码库级 Docker 环境。
- pre-patch 与 post-patch 可执行文件。
- 生成并清洗后的 target vulnerability description。
- 原始 PoC、崩溃类型 / 栈和 ground-truth patch。
- 四档信息设置，用于控制是否泄露文件、行号或补丁信息。

9.3 真值、评分与质量控制

Agent 提交的 PoC 在 pre-patch 版本必须触发目标崩溃，在 post-patch 版本不得触发。该动态差分比字符串匹配更接近真实漏洞复现，并允许 Agent 产生与参考 PoC 不同的输入。

- 修复前 / 后使用相同构建配置，减少环境差异。
- 崩溃栈和 sanitizer 类型用于排除无关异常。
- 可调信息级别区分从描述推理与从补丁逆向触发的难度。
- 数据规模统计显示项目中位数约 1,117 个文件、387,491 行代码，补丁中位数为 1 个文件、7 行。

9.4 构造偏差与使用限制

样本由 fuzzing 发现并要求可由 sanitizer 观察，因此强烈偏向 C/C++ 内存安全和输入解析问题。任务给定漏洞描述，通常不要求真正从零审计攻击面；差分 oracle 也可能把补丁后的 sibling bug 或不稳定崩溃处理得过于简单。

- 对业务逻辑、认证授权、Web 状态机和并发分布式漏洞覆盖较弱。
- 生成描述可能泄露函数名或触发条件，影响零日真实性。
- 公开 PoC、补丁和 issue 造成显著数据污染风险。

对训练与评测设计的启示

自动化构造大规模真实漏洞 benchmark 的可行路径是：固定时间窗口、二分定位修复、双版本重建、动态差分判分。但该路径天然受原始漏洞发现渠道的分布约束。

主要依据：[CyberGym: Evaluating AI Agents' Real-World Cybersecurity Capabilities at Scale](#)

10. ExploitBench

面向 V8 的 16 级利用能力阶梯 benchmark

论文 / 资料版本	arXiv:2605.14153, 2026-05
任务定位	在生产级缓解环境中把利用过程拆成覆盖、触发、原语、控制流和代码执行等可测能力。
数据规模	41 个 2024 年及以后披露的 V8 N-day；每个漏洞构建多种脆弱 / 修复二进制。
样本基本单位	一个 V8 漏洞与一组 16 项能力 flags 的组合。
证据等级	A：论文 + benchmark 网站 / 环境。
一句话理解 ExploitBench 不把“crash”当作最终成功，而是为同一个 exploit 轨迹点亮 16 个能力旗标，从到达脆弱代码一直量到任意代码执行。	

10.1 数据构造流水线

- 1

V8 漏洞选择：从上游 issue tracker 与 Gitiles 收集 2024 年及以后披露的 V8 N-day，要求具备明确 vulnerable / fix commits，并符合 v8CTF 奖励资格。最终选择 41 个真实漏洞。
- 2

可复现工具链固定：固定 Debian 镜像 digest、apt snapshot、depot_tools、V8 源码与 DEPS，避免依赖随时间漂移。每个实例镜像保留完整源码历史与补丁上下文，单镜像体积可达约 80 GB。
- 3

多二进制构建：为脆弱版本构建 debug、debug-ASan、release、release-ASan、coverage 等 5 个二进制，为修复版本构建 4 个对照二进制，以支持覆盖、差分、崩溃和生产利用多种 oracle。
- 4

任务输入设计：Agent 获得 bug ID、简短描述、patch diff 和 grader builtins，但不获得参考 PoC。环境允许修改工作副本，最终评分始终使用未被 Agent 修改的 ground-truth 二进制。
- 5

能力阶梯定义：将利用拆成 16 个可单独验证的 flags，覆盖执行到目标代码、差分行、触发崩溃、跨越 sandbox / cage、构造读写原语、控制程序计数器以及任意代码执行等层级。
- 6

重复执行与审计：每个 grade 在 3 个随机轮次、2 个二进制和 2 个配置上共执行 12 次，并要求一致通过。构造者还进行 11 类离线作弊审计和人工抽查。

10.2 落盘对象与数据格式

ExploitBench 的数据对象不仅是漏洞镜像，还包含能力判定逻辑和随机挑战。它为每个漏洞保存多种构建，使早期能力与最终 exploit 可在不同插桩条件下独立验证。

- 41 个 V8 漏洞的源码、描述、patch diff 与固定依赖。
- 5 个 vulnerable binaries 和 4 个 fixed binaries。
- 16 项 capability flags 及其确定性 oracle。
- 每次运行随机生成的 challenge-response secret。
- coverage、differential、sandbox primitive、read / write、PC control、ACE 等证据。

10.3 真值、评分与质量控制

grader 使用未修改的 ground-truth binaries，结合差分执行和随机挑战验证每项能力。对任意读写等原语，Agent 必须读取或写入当次随机秘密；对代码执行，使用 signal-handler proof，而不是接受打印字符串。

- 每项能力在 12 个执行组合上要求一致，降低偶然性。
- 随机秘密使硬编码 flag 或重放固定输出无效。
- 修复二进制用于排除与目标漏洞无关的 V8 行为。
- 多层 flags 提供比单一 RCE 更密集、可解释的训练信号。

10.4 构造偏差与使用限制

只覆盖 V8，且 Agent 获得 patch diff，因此主要测量 N-day 利用推理而非漏洞发现。41 个样本规模不大，V8 漏洞类型和 heap sandbox 机制也难以直接外推到内核、原生程序或 Web。

- 论文对“所有候选到 41 个最终样本”的完整筛选漏斗披露有限。
- 巨型镜像和多次运行导致计算、存储成本很高。
- 能力 flags 虽细，但不同漏洞可达到的最高能力并不完全相同。

对训练与评测设计的启示

高级利用 benchmark 应把 crash 后的关键能力显式拆开，并用随机挑战验证真实原语。这样的 reward ladder 比“是否弹 shell”更适合诊断模型卡点，也更适合课程式训练。

主要依据：[ExploitBench: A Capability Ladder Benchmark for LLM Cybersecurity Agents](#)

11. CVE-Bench (Web 利用版)

40 个关键严重度真实 Web CVE 的沙箱化利用 benchmark

论文 / 资料版本	arXiv:2503.17332, ICML 2025; 注意与同名漏洞修复 benchmark 区分
任务定位	在接近真实部署的 Web 应用中, 根据零日或一日信息完成可验证安全影响。
数据规模	40 个 CVSS v3.x >= 9.0 的关键 Web CVE, 来源时间窗口为 2024-05-01 至 2024-06-14。
样本基本单位	一个 CVE 的应用、数据库 / 辅助服务、参考攻击目标和动态 grader。
证据等级	A: 论文 + benchmark 框架; 参考自动化 exploits 并非全部公开。
一句话理解 CVE-Bench 先从一个固定 NVD 时间窗完整收集 critical CVE, 再经过可部署性与可复现性筛选, 把最终 40 个 Web 漏洞重建为状态型动态任务。	

11.1 数据构造流水线

1

固定窗口全量收集: 抓取 NVD 在 2024 年 5 月 1 日至 6 月 14 日发布、CVSS v3.x 标记为 CRITICAL 的条目, 避免仅凭作者偏好挑选著名漏洞。

2

双人纳入筛选: 两名作者检查候选是否属于开源 Web 应用、能跨平台部署、可在隔离环境复现, 并且利用过程不需要把敏感数据发送给外部真实服务。初步有 60 个条目通过。

3

复现失败排除: 排除无法获得脆弱版本、必须串联另一个未被 benchmark 支持的漏洞、只有过短 NVD 描述且研究者无法复现的条目, 最终保留 40 个。

4

应用与依赖容器化: 为每个 CVE 构建应用、数据库和辅助服务容器, 初始化必要账号与数据, 恢复漏洞版本。单个实例通常投入 5-24 人时。

5

参考 exploit 与目标归一化: 根据公开 PoC, 或结合描述与 patch 手工恢复参考攻击; 再把不同漏洞归一为 8 类标准安全目标, 例如拒绝服务、读取 / 创建文件、读取 / 修改数据库、未授权管理员登录、提权和服务端外连请求。

6

零日 / 一日信息视图: 零日设置不提供 CVE 描述, 一日设置提供 NVD 描述; 环境和漏洞相同, 仅改变 Agent 可见信息, 以区分发现与已知漏洞利用能力。

11.2 落盘对象与数据格式

每个实例是完整可启动的 Web 系统，而非单个漏洞代码片段。grader 以应用状态或可观察影响判分。

- 漏洞应用、数据库、依赖服务和网络配置。
- 脆弱版本与初始化数据。
- 零日 / 一日两种任务说明。
- 8 类攻击目标及每应用专用 evaluator。
- 用于构造验证的参考 exploit；部分仅按申请提供。

11.3 真值、评分与质量控制

动态 evaluator 持续检查目标系统的状态变化，而不是要求 payload 与参考 exploit 相同。例如检查特定文件、数据库行、管理员会话或服务端请求。对 XSS 等需要受害者交互的漏洞，环境会模拟浏览器 / 用户。

- 状态型 grader 允许不同攻击路径达到相同安全影响。
- 隔离网络防止 exploit 访问真实第三方系统。
- 参考攻击在构造阶段确保漏洞可复现，评测阶段不要求 exact match。
- 固定时间窗和明确排除规则使样本选择较透明。

11.4 构造偏差与使用限制

仅选 CVSS ≥ 9.0 且在短时间窗公开的 Web CVE，不能代表常见中低危漏洞或其他年份。20 个被排除候选中包含环境难以重建的漏洞，因此最终数据偏向容易容器化、依赖较少的项目。

- 参考 exploit 未完全公开，第三方难以全面审计构造真值。
- NVD 描述和公开 PoC 带来训练污染。
- 本文使用的是 Web exploitation 版 CVE-Bench，不应与同名修复类 benchmark 混用。

对训练与评测设计的启示

构造真实 Web CVE benchmark 时，先定义统一安全影响，再为每个应用写状态型 grader，比统一要求某种 payload 更稳健。固定公开时间窗和记录所有排除原因也能减少 cherry-picking。

主要依据：[CVE-Bench: A Benchmark for AI Agents' Ability to Exploit Real-World Web Application Vulnerabilities](#)

12. AutoPenBench

课程型合成靶场与真实 CVE 混合的自动渗透测试 benchmark

论文 / 资料版本	arXiv:2410.03225v2, 2024-2025
任务定位	在 Kali 容器中完成多步侦察、利用和取旗，并用里程碑衡量部分进度。
数据规模	33 个 Docker 任务：22 个 in-vitro 人工设计任务 + 11 个真实 CVE 任务。
样本基本单位	一个目标容器或小型容器组，以及一条人工 gold path 和若干 milestone。
证据等级	A：论文 + AgentQuest / AutoPenBench 代码。
一句话理解 AutoPenBench 用 22 个教学靶题覆盖基础技术，再用 11 个 Metasploit 可利用 CVE 增加真实性；它的特色是为每题手工写 gold steps 并用 LLM 判断 Agent 完成了多少中间步骤。	

12.1 数据构造流水线

- 1

框架基础：建立在 AgentQuest 任务框架上，为每个任务提供可启动环境、Agent 工作站、目标说明和结果收集接口。
- 2

In-vitro 任务手工设计：根据网络安全课程场景人工设计 22 个脆弱或错误配置容器，覆盖访问控制 5、Web 安全 7、网络安全 6、密码学 4。每题目标均能通过一条明确的教学路径完成。
- 3

真实 CVE 选择：收集 2014-2024 年公开漏洞，要求存在可用的 Metasploit exploit module，以降低历史漏洞复现失败率。最终加入 11 个真实 CVE 环境。
- 4

网络与工具环境：Agent 在 Kali Linux 工作站容器内操作，可调用常见安全工具和 shell；目标容器位于一个 /16 Docker 网络。利用成功后可以访问 secret flag。
- 5

Gold steps 编写：构造者为每题手工写一条高效参考解法，将其拆为命令级 milestone 和阶段级 milestone，例如扫描、服务识别、漏洞利用、权限提升和读取目标。
- 6

轨迹判分：将 Agent 日志逐个与 milestone 对比，由 GPT-4o 判断某个目标步骤是否完成；模型判断不可靠时允许人工修正。Progress Rate 由完成的命令 milestone 数除以总 milestone 数得到。

12.2 落盘对象与数据格式

任务目录主要包含 Docker 配置、目标服务、flag、参考 gold steps 和里程碑。其数据结构更接近可教学的渗透实验，而不是大量真实仓库。

- Kali workstation 与目标 Docker 容器。
- 22 个合成课程任务和 11 个真实 CVE 任务。
- 任务说明、secret flag 和网络配置。
- 人工 gold steps、command milestones、stage milestones。
- Agent 轨迹与 GPT-4o / 人工判定结果。

12.3 真值、评分与质量控制

最终完成由 flag 判断；部分进度由 LLM 将轨迹与单个 milestone 对齐。逐个 milestone 提问而不是让模型一次总结整条轨迹，可以减少遗漏；人工可对幻觉或错误匹配进行修正。

- flag 提供确定性终局信号。
- 里程碑提供长任务的稠密反馈。
- gold path 是一条参考路径，不保证覆盖所有等价解法。
- LLM judge 与人工修正使评分具有可解释性，但降低完全自动化程度。

12.4 构造偏差与使用限制

22 / 33 任务是教学型人工环境，真实度有限；11 个 CVE 因要求已有 Metasploit module，偏向著名、工具化程度高的漏洞。里程碑依据单条 gold path，可能漏记创新但有效的攻击路径。

- 网络拓扑较简单，不等价于复杂企业后渗透。
- LLM milestone judge 会随模型版本变化。
- 课程类别计数不等同于真实漏洞分布。

对训练与评测设计的启示

里程碑奖励适合早期训练，但构造时应允许多条等价路径，或尽量将 milestone 定义为环境状态，而非固定命令；否则 Agent 使用不同工具就可能被低估。

主要依据：[AutoPenBench: Benchmarking Generative Agents for Penetration Testing](#)

13. CyBench

从四个专业 CTF 比赛精选、并补充子任务的 40 题 benchmark

论文 / 资料版本	arXiv:2408.08926v4, ICLR 2025
任务定位	评估通用安全推理、工具调用和多步解题能力，并用人类首解时间覆盖难度谱。
数据规模	40 题，来自 HackTheBox Cyber Apocalypse、SekaiCTF、GlacierCTF、HKCert 四个比赛。
样本基本单位	一个可启动 CTF challenge，含题面、附件、flag evaluator 和人工子任务。
证据等级	A：论文 + GitHub / 网站。
一句话理解 CyBench 不是把某个比赛全量打包，而是先给比赛整体打质量分，再精选 40 个近期专业题，并以 First Solve Time 构造从数分钟到近 25 小时的难度分布。	

13.1 数据构造流水线

1

比赛质量筛选：分析 2022-2024 年近期专业 CTF，对比赛的可运行题目比例、部署难度、官方解法 / writeup 完整性进行评分，并要求公开 GitHub 发布，以保证能够合法重建。

2

题目精选与领域覆盖：从 HackTheBox Cyber Apocalypse 2024 选 17 题、SekaiCTF 2022-2023 选 12 题、GlacierCTF 2023 选 9 题、HKCert 选 2 题；类别为 crypto 16、web 8、reverse 6、forensics 4、misc 4、pwn 2。

3

人类难度标定：使用比赛 First Solve Time 作为难度代理，按近似对数尺度选择从 2 分钟到 24 小时 54 分钟的题。不同比赛的 FST 从排行榜、Discord 首解公告或队伍时间戳手工恢复。

4

统一任务规范：每题标准化为 description、starter files、local / remote 服务和 evaluator。Agent 在通用 Kali Linux 环境中执行命令，远端服务位于隔离容器网络。

5

人工子任务分解：作者依据官方解法或 writeup 把每题拆成顺序中间问答，使尚不能完全拿 flag 的模型也能获得部分分；子任务提供引导版与无引导版两种评测视角。

6

可运行性和泄漏检查：为每题编写 solution.sh，CI 比较最终输出与参考 flag，并检查服务存活。作者人工检查 Agent 轨迹，发现通过 Docker 缓存、文件泄漏等非预期捷径后进行隔离和修复。

13.2 落盘对象与数据格式

CyBench 以统一任务接口包装来自不同比赛的异构题目，并增加 FST 与子任务元数据。

- 40 个 challenge 的题面、附件、服务 Docker 配置。
- 本地与远端任务接口、参考 flag 和 evaluator。
- 人工 solution.sh 与官方 / 公开 writeup。
- 人类 First Solve Time 和比赛来源。
- 逐题人工子任务，用于 guided partial-credit 评测。

13.3 真值、评分与质量控制

完整任务通过提交正确 flag 判分；子任务按人工答案判定中间步骤。CI 使用参考 solution.sh 验证任务仍可运行，人工轨迹审查用于发现 flag 泄漏和非预期主机访问。

- FST 比题目分值更接近真实人类解题难度，但采集方式跨比赛不一致。
- 远端服务与 Agent 工作站隔离，降低直接读取 flag 的风险。
- 子任务使失败轨迹可诊断，但也向模型泄露解题路径。
- 公开题目和 writeup 容易被模型记忆，论文因此强调近期性和多比赛来源。

13.4 构造偏差与使用限制

CTF 是人为设计的封闭问题，通常比真实仓库更小、更干净，且成功标准固定为 flag。40 题的类别分布偏重密码学、pwn 较少；FST 受参赛队水平、题目分批发布和比赛规则影响。

- 公开 writeup 带来明显 contamination。
- guided 子任务成绩不应与无提示完整 solve rate 混合。
- 同一比赛题的工具链和设计风格相关，样本不独立。

对训练与评测设计的启示

精选小规模 benchmark 时，应公开来源比赛、题目选择标准和人类难度代理；同时保留无引导完整任务与有引导子任务两套结果，避免部分分掩盖端到端能力。

主要依据：[Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models](#)

14. NYU CTF Bench

CSAW CTF 2017-2023 全面归档与容器化的 200 题开放数据集

论文 / 资料版本	arXiv:2406.05590v3, NeurIPS 2024 Datasets and Benchmarks; 当前仓库含 200 test + 55 development
任务定位	提供可扩展、可重复部署的多类别 CTF 语料, 用于 Agent 开发与完整测试。
数据规模	主测试集 200 题, 开发集 55 题; 覆盖 CSAW Quals / Finals 的六个类别。
样本基本单位	一个 CSAW challenge 目录, 含 challenge.json、可见文件、可选服务容器和隐藏 flag。
证据等级	A: 论文 + GitHub 数据仓库。
一句话理解 NYU CTF Bench 的主要工作不是精选 40 道代表题, 而是把多年 CSAW 题目批量恢复、补齐依赖并统一成可由 Agent 自动启动的开放数据库。	

14.1 数据构造流水线

1

历史题库汇集: 从 2017-2023 年 CSAW CTF Quals 与 Finals 汇集公开挑战, 覆盖 crypto、web、pwn、reverse、forensics、misc。论文不同段落对初始池写有 568 与 567, 其他公开材料还出现 528, 属于文献内部不一致。

2

人工可运行性验证: 从历史池中人工验证 200 题, 检查题目是否能部署、依赖包是否仍可获得、服务协议和答案是否在当前环境仍有效。难以恢复的题不进入主 benchmark。

3

服务题网络修复: 对 server challenge 检查 Docker 网络内外连通, 修复端口、host、容器依赖和外部服务失效问题, 使 LLM 自动化框架可以通过统一接口访问。

4

非服务题制品补齐: 检查源码、配置、二进制、压缩包、图像、PCAP 等附件是否完整, 定位缺失文件并将必要依赖封装进容器。

5

目录与元数据标准化: 统一目录为 <year>/<event>/<category>/<challenge>。每题提供 challenge.json, 记录题名、描述、向模型可见的 files、host / internal ports 和隐藏 ground-truth flag。

6

测试 / 开发划分: 当前仓库把 200 题置于 test, 并额外提供 55 题 development, 可作为 Agent 工具和提示设计的训练 / 开发集, 减少针对测试题反复调参造成的偏差。

14.2 落盘对象与数据格式

数据集保留原始题目制品，同时通过 JSON 控制 Agent 可见范围。服务题含 docker-compose，非服务题也可将依赖放入容器，便于统一调度。

- 200 个 test challenges 和 55 个 development challenges。
- 按年份、Quals / Finals、类别和题名组织的目录。
- challenge.json: 题面、可见附件、网络参数和隐藏 flag。
- Docker Compose、原始 Dockerfile、服务源码和配置。
- 二进制、媒体、PCAP、磁盘镜像等异构附件。

14.3 真值、评分与质量控制

主要 oracle 是隐藏 flag。数据构造阶段通过人工求解和启动检查确认题目可解；部署脚本确保 server challenge 可从 Agent 网络访问。难度采用原作者给出的 1-500 分值，而不是统一的人类时间。

- JSON 的 files 列表决定模型可见附件，避免把服务端源码或 flag 意外暴露。
- 容器化降低历史包更新导致的不可运行问题。
- development split 为 Agent 设计提供公开调试样本。
- 当前仓库规模应以 200 test + 55 development 为准。

14.4 构造偏差与使用限制

数据只来自 CSAW 一个赛事系列，题目风格和作者群体集中；分值是主观难度，跨年份不完全可比。公开题目、源码和 writeup 存在强 contamination；部分难以验证的 incident response 类题被排除。

- 论文对初始候选池数量存在 568 / 567 等不一致，报告应避免引用为精确无争议事实。
- 200 题的类别分布不均衡，不能用总 solve rate 代替分领域分析。
- CTF flag 仍无法衡量真实系统中的稳健性、隐蔽性和影响范围。

对训练与评测设计的启示

大规模归档型 benchmark 的核心是“维护工程”：显式记录可见文件、网络参数、隐藏真值和依赖，并提供开发 / 测试分割。仅把历史题目链接汇总在一起并不足以形成可重复评测。

主要依据：[NYU CTF Bench: A Scalable Open-Source Benchmark Dataset for Evaluating LLMs in Offensive Security](#)

15. 跨 benchmark 的构造方法总结

15.1 一个高质量真实漏洞实例至少需要七类对象

- 1 **来源对象**：CVE / advisory / issue / bounty / fuzz report，以及完整来源和时间戳。
- 2 **版本对象**：vulnerable、fixed，必要时 latest；明确 commit、依赖和工具链。
- 3 **入口对象**：真实用户入口、协议、文件格式、syscall 或服务 URL，避免只暴露内部 harness。
- 4 **攻击证据**：PoC / PoV / exploit、真实 stderr、崩溃签名、应用状态或主机证明。
- 5 **修复证据**：linked patch、补丁版本、功能回归测试和安全不变量。
- 6 **环境对象**：容器 / 虚拟机、数据库、网络、账号、凭证、缓解机制和资源预算。
- 7 **评分对象**：动态 oracle、归因规则、重试策略、随机秘密、抗作弊边界和人工复核记录。

15.2 最常见的构造失败模式

只验证修复前崩溃
Agent 可能触发无关 bug、资源耗尽或测试专用 assert。至少要增加 fixed，复杂系统最好再增加 latest。
只保存参考 patch
不能判断 Agent patch 是否删除功能、拒绝所有输入或只特判 PoC。需要项目功能测试与业务不变量。
把命令当作里程碑
不同工具和等价攻击路径会被漏记。优先把里程碑写成环境状态或获得的能力。
公开数据无时间切分
模型可能背诵报告和 writeup。应记录披露时间，并提供未来持续更新或 sealed test。
只给最终稀疏奖励
长程任务难以训练。可以引入入口定位、路径可达、PoC、影响、修复等分层信号，但每层都必须抗作弊。

忽略环境漂移

历史依赖、镜像和外部服务会失效。必须固定 digest、包快照、源码 DEPS，并在 CI 中定期重新执行参考解。

15.3 面向 Coding Agent 网络安全专项的组合建议

没有单个公开 benchmark 能同时完整覆盖真实漏洞发现、复杂利用、补丁质量和内网攻击链。更合理的评估套件是按能力分层组合，并对同一模型分别报告各层结果。

仓库审计与 PoC	CyberGym-E2E + SEC-bench Pro + VulnGym。前两者提供动态 PoC，后者补充入口与代码路径解释。
Web 漏洞生命周期	BountyBench + CVE-Bench。前者加入 Detect / Patch 和经济影响，后者提供标准化 Web 安全影响。
高级利用	ExploitGym + ExploitBench。前者规模和领域广，后者 V8 能力阶梯细。
渗透与后渗透	AgentCyberRange + AutoPenBench。前者用于真实多主机链，后者适合轻量训练和里程碑调试。
通用安全基础	CyBench + NYU CTF Bench。CyBench 适合小而难的回归，NYU 适合大规模分领域统计和 Agent 开发。
推荐的统一主指标 端到端通过率之外，应至少同时报告：漏洞发现 precision / recall、PoC 可重现率、目标漏洞归因准确率、利用能力最高等级、补丁安全通过率、功能回归通过率、平均工具调用 / 时间 / 成本，以及按项目和漏洞家族分组后的宏平均。	

16. 参考文献与项目链接

- [1] [CyberGym-E2E](#)
- [2] [BountyBench](#)
- [3] [SEC-bench Pro v2](#)
- [4] [ExploitGym](#)
- [5] [AgentCyberRange](#)
- [6] [VulnGym repository](#)
- [7] [CyberGym](#)
- [8] [ExploitBench](#)
- [9] [CVE-Bench \(Web exploitation\)](#)
- [10] [AutoPenBench](#)
- [11] [CyBench](#)
- [12] [NYU CTF Bench](#)
- [13] [NYU CTF Bench repository](#)

版本说明

所有规模 and 状态以 2026 年 7 月 28 日可核验的论文版本与公开仓库为准。后续论文修订、数据增补或仓库删除可能改变数字；特别是 SEC-bench Pro 与 VulnGym 应在再次使用前检查版本。